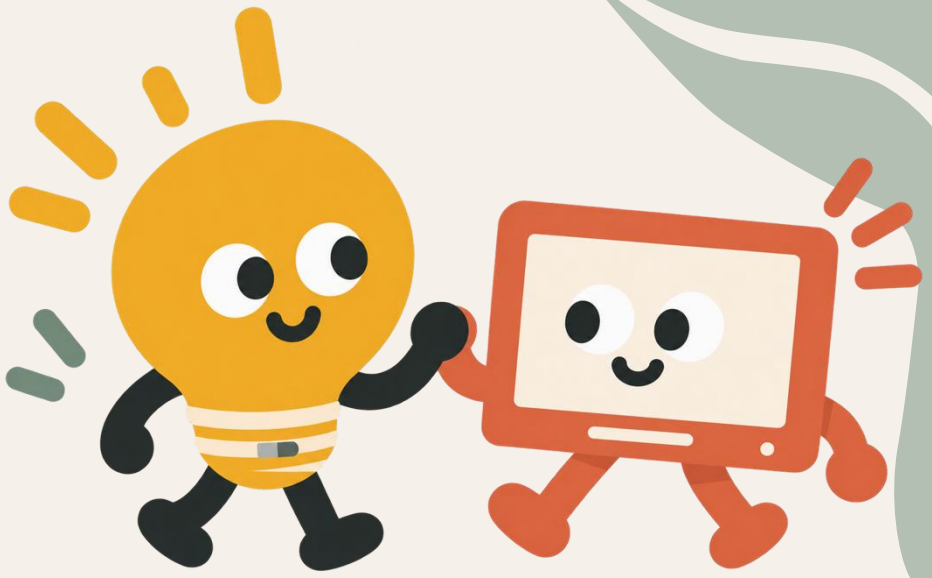
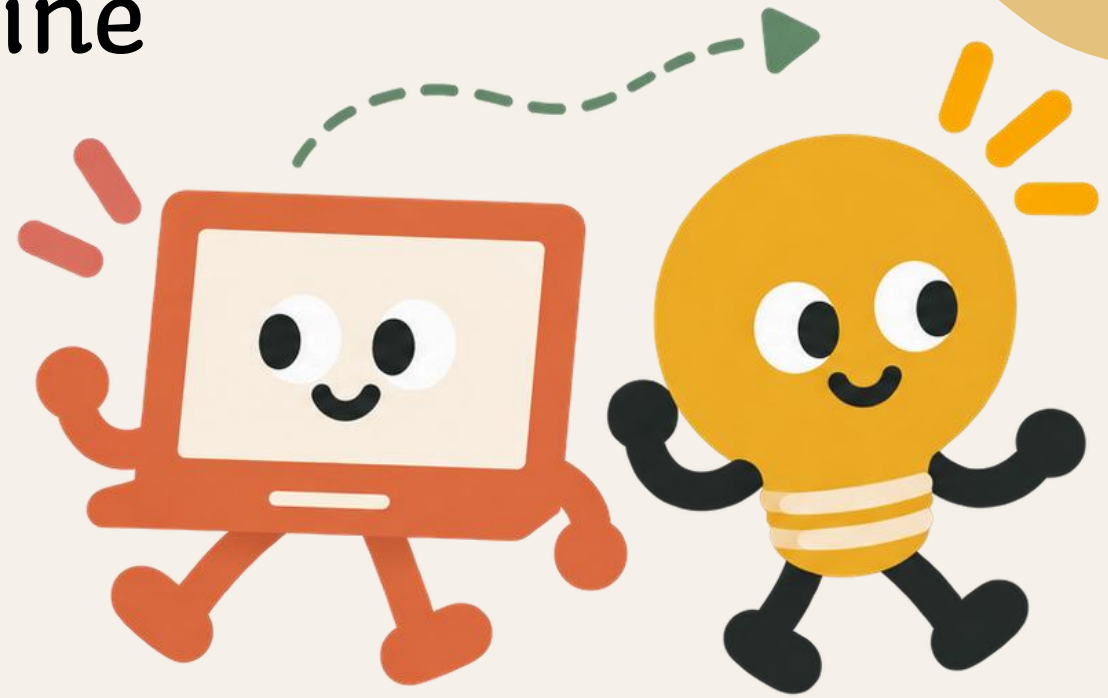
Módulo 4

Taller teórico-práctico
Del dataset a U-Net: segmentación de imágenes con Deep Learning

→ Diseño experimental y pipeline de datos



Construcción del pipeline de datos



¿Qué es un pipeline de datos?

Es el conjunto de pasos que sigue cada muestra desde que está almacenada en el disco hasta que llega al modelo para el entrenamiento o la evaluación.

El pipeline se encarga de preparar y entregar los datos en el formato correcto, de manera reproducible y eficiente.



1 Manifest + split

Definimos qué imágenes pertenecen a cada conjunto (train/val/test).

ruta_imagen	ruta_máscara	split
img_001.jpg	mask_001.png	train
img_002.jpg	mask_002.png	train
img_003.jpg	mask_003.png	val
img_004.jpg	mask_004.png	test
...	...	...

2 Dataset: localizar y leer

A partir de una fila del manifest, localiza y lee la imagen y su máscara desde el disco.



Imagen (leída del disco) | Máscara (leída del disco)

3 Transformar imagen y máscara

Aplica las mismas transformaciones (p. ej. resize, augmentations) de forma sincronizada a la imagen y a su máscara.



Imagen (transformada) | Máscara (transformada)

4 Convertir a tensores

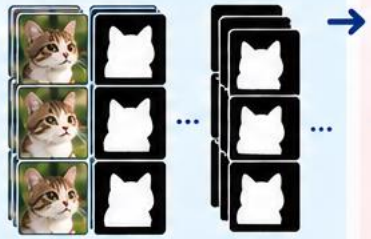
Convierte la imagen y la máscara al formato de tensores (por ejemplo, PyTorch).



Tensor imagen [C, H, W] (p. ej. [3, 256, 256]) | Tensor máscara [H, W] (p. ej. [256, 256])

5 DataLoader: agrupar en batches

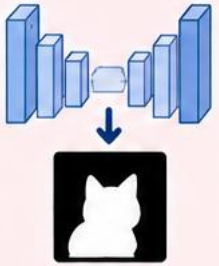
Organiza cómo se solicitan y agrupan las muestras en lotes (batches) para el entrenamiento o la evaluación.



Batch de imágenes [batch_size, C, H, W] | Batch de máscaras [batch_size, H, W]

6 Modelo U-Net

Recibe los batches de imágenes y produce una predicción de segmentación.



Predicción [batch_size, H, W]

Dataset

Define cómo acceder y preparar una muestra individual.

- Lee las rutas desde el manifest.
- Localiza y carga la imagen y la máscara.
- Aplica las transformaciones.
- Convierte a tensores.
- Devuelve una sola muestra (imagen, máscara).

Una muestra



image: [C, H, W]
mask: [H, W]

DataLoader

Organiza cómo se solicitan y agrupan las muestras.

- Recibe el dataset.
- Solicita varias muestras.
- Forma lotes (batches).
- Puede mezclar el orden (shuffle).
- Puede usar varios procesos para cargar los datos (num_workers).

Un batch



images: [B, C, H, W]
masks: [B, H, W]

Analogía

Dataset (prepara un elemento)

DataLoader (organiza los lotes)



Clase Dataset para segmentación

La clase **Dataset** define cómo acceder a una muestra del dataset y cómo prepararla.
Debe heredar de **torch.utils.data.Dataset** y sobrescribir los métodos **__init__**, **__len__** y **__getitem__**.

El Dataset se encarga de preparar una muestra individual. El **DataLoader** después se encargará de organizar las muestras en lotes (batches).



Código base

```python
from torch.utils.data import Dataset

class SegmentationDataset(Dataset):

    def __init__(self, manifest):
        self.manifest = manifest.reset_index(drop=True)

    def __len__(self):
        return len(self.manifest)

    def __getitem__(self, idx):
        row = self.manifest.iloc[idx]
        return row["image_path"], row["mask_path"]
```

1 __init__ (inicialización)

Se ejecuta al crear el objeto de la clase.

Guarda las referencias y configuración necesarias.

manifest (DataFrame)

image_path	mask_path
img_001.jpg	mask_001.png
img_002.jpg	mask_002.png
img_003.jpg	mask_003.png
...	...

→

self.manifest

image_path	mask_path
img_001.jpg	mask_001.png
img_002.jpg	mask_002.png
img_003.jpg	mask_003.png
...	...

En este ejemplo se guarda el DataFrame del manifest y se reinicia el índice para tener un índice consecutivo desde 0.

2 __len__ (longitud)

Se ejecuta cuando se consulta el tamaño del dataset (por ejemplo, con `len(dataset)`).

Devuelve el número total de muestras que contiene el dataset.

`return len(self.manifest)` → **Número de muestras del dataset**: **3** (en este ejemplo)

Es necesario para que el DataLoader sepa cuántas muestras hay disponibles y pueda iterar sobre ellas.

3 __getitem__ (obtener una muestra)

Se ejecuta cada vez que se solicita un índice (por ejemplo, dataset[0]).

Recupera la información de una muestra (y en la práctica suele leer y preparar la imagen y la máscara).

`idx = 0` →

image_path	mask_path
img_001.jpg	mask_001.png

→ **Imagen**  **Máscara** 

En este ejemplo simplemente se devuelven las rutas.
En un caso completo, aquí se leerían las imágenes, se aplicarían las transformaciones y se convertirían a tensores.



El **Dataset** se encarga de una muestra a la vez. El **DataLoader** pedirá varias de estas muestras y las agrupará en batches.

Resumen de los métodos

Método	¿Cuándo interviene?	Función	¿Qué ocurre en este ejemplo?
__init__	Al crear el objeto	Guardar referencias y configuración	Se guarda el DataFrame del `manifest` y se reinicia el índice (`reset_index`).
__len__	Al consultar su longitud (p. ej. len(dataset))	Informar cuántas muestras contiene	Devuelve el número de filas del `manifest` (`len(self.manifest)`).
__getitem__	Al solicitar un índice (p. ej. dataset[0])	Recuperar y preparar una muestra	Obtiene la fila correspondiente y devuelve las rutas de la imagen y la máscara (`row["image_path"]`, `row["mask_path"]`).

Introducción a DataLoader

El **DataLoader** se encarga de solicitar las muestras al Dataset y agruparlas en lotes para el entrenamiento o la evaluación.

El DataLoader no modifica las imágenes ni las máscaras, solo organiza cómo se solicitan y se agrupan.



Código

```python
from torch.utils.data import DataLoader

train_loader = DataLoader(
    train_dataset,
    batch_size=8,
    shuffle=True,
    num_workers=0
)
```

Parámetros principales

Parámetro	Qué controla	Detalles
`batch_size`	Número de muestras por lote.	• Define cuántas imágenes (y máscaras) se agrupan en cada batch. • El último batch puede ser más pequeño si el número total de muestras no es múltiplo del batch_size.
`shuffle`	Si se reordena la secuencia de muestras al crear cada iteración.	• Mezcla el orden de las muestras en cada época (entrenamiento). • No modifica la asignación a train, validation o test.
`num_workers`	Número de procesos utilizados para cargar las muestras.	• num_workers=0: la carga se realiza en el proceso principal. • Valores > 0: permiten carga paralela. • No siempre mejora el rendimiento (depende del equipo y del tipo de datos). • **Para el taller se recomienda usar num_workers=0 (especialmente con Jupyter).**

1 batch_size

Número de muestras por lote.

Ejemplo: 20 imágenes con batch_size=8



2 shuffle

Reordena la secuencia de muestras en cada iteración.

Las mismas imágenes, pero en un orden diferente en cada época de entrenamiento.



El **shuffle** solo cambia el orden dentro del conjunto de entrenamiento. No modifica la asignación a train, validation o test.

3 num_workers

Número de procesos para cargar las muestras.

num_workers = 0

- La carga de datos se realiza en el proceso principal.
- Más sencillo y estable (especialmente en Jupyter).
- **Recomendado para el taller.**



Carga secuencial en un solo proceso

num_workers > 0

- Permite la carga paralela de datos usando varios procesos.
- Puede acelerar la lectura de datos, especialmente en datasets grandes.
- No siempre mejora el rendimiento (depende del equipo y del tipo de datos).



Carga en paralelo con varios procesos